

0.65.2. Une lettre qui avance — doublé, deux positions

Le A et le B avancent ensemble : la même x, deux lignes.

1. Ce qu'il faut comprendre

C'est le 0.65 et le 0.65.1 réunis : deux `effacer`, deux `poser`, la même `x`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1441 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 0;           // la colonne des deux lettres
```

2. UNE VARIABLE — IMAGES

```
uint8_t images = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

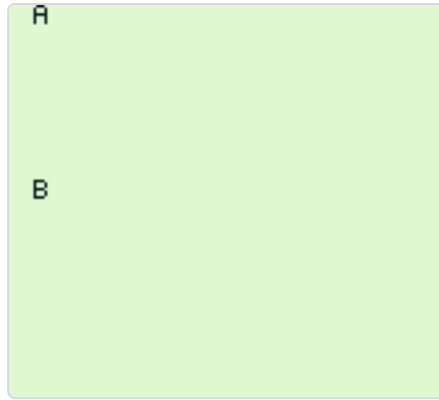
```
int main() {
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            effacer(x, 0, 1);
            effacer(x, 8, 1);
            x++;
            if (x == 20) x = 0;
        }
        poser(x, 0, ALPHABET[0]); // le A, ligne 0
        poser(x, 8, ALPHABET[1]); // le B, ligne 8
    }
}
```

3. Ce qui se passe, étape par étape

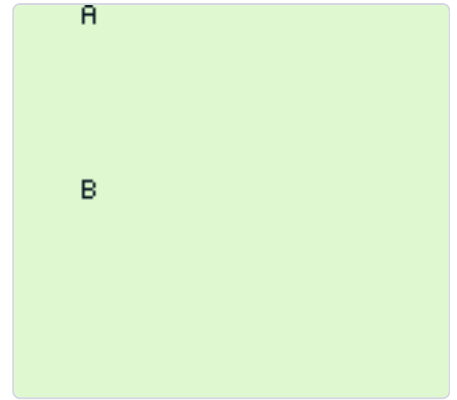
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



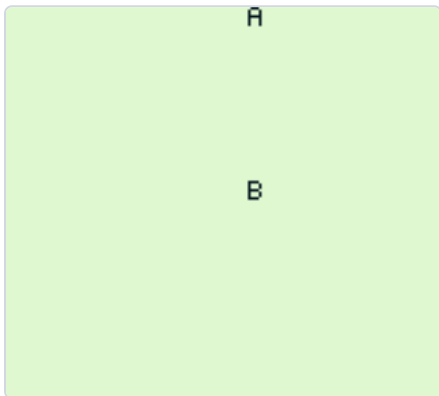
la toute première image — la console vient de démarrer — image 1



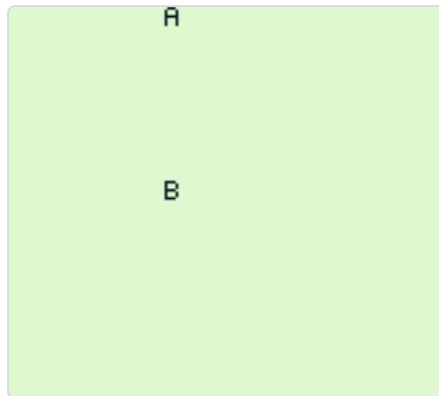
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A et le B avancent ensemble, sur la ligne 0 et sur la ligne 8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A et le B à la même colonne ($x = 5$)

Ensuite : **0.65.3. Une lettre qui avance — en simple** — niveau 0 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++